Verification-First Clinical Engineering

A Zero-Trust Pipeline for Deployment-Ready MOH Guidelines

Current digitization is a manual, error-prone translation task

- **The Status Quo:** Developers interview MDs or interpret Ministry of Health (MOH) PDFs to write code (CHT, ODK, OpenSRP).
- **The Risk:** "Trust-based" transcription introduces silent errors:
  - Missed exclusion criteria (e.g., age cutoffs).
  - Unsafe defaults (e.g., treating missing temperature as "normal").
  - Logic gaps (e.g., stockout scenarios ignored).
- **Code takes years to write!** So few CHWs have clinical support on their phones

The Solution: Zero-Trust Engineering

Core Philosophy: "Zero-Trust." We assume every artifact is flawed until proven otherwise

The Verification Stack:

- **Adversarial Extraction:** "Red Teams" hunting for logic gaps at each step of creating the clinical logic and code
- **Formal Verification:** Mathematical proofs of desirable properties: no infinite loops, decision trees have graceful endpoints, etc.
- **Human Sovereignty:** Mandatory sign-offs of clinical logic & of translations
- **Traceability:** Cryptographic links to source text
- **Differential Testing:** Synthetic patient simulations show approved clinical logic operates the same as the final code

The Solution: Zero-Trust Engineering

Core Philosophy: "Zero-Trust." We assume every artifact is flawed until proven otherwise

The Verification Stack:

- **Adversarial Extraction:** "Red Teams" hunting for logic gaps at each step of creating the clinical logic and code
- **Formal Verification:** Mathematical proofs of desirable properties: no infinite loops, decision trees have graceful endpoints, etc.
- **Human Sovereignty:** Mandatory sign-offs of clinical logic & of translations
- **Traceability:** Cryptographic links to source text
- **Differential Testing:** Synthetic patient simulations show approved clinical logic operates the same as the final code

Validating our approach

- Evidence
  1. **Independent Oracles:** Validation against external ground truths.

Adversarial Logic Extraction

Replacing "Single-Pass Coding" with "Red Teaming"

- **Multi-Pipeline:** We run **two independent** AI pipelines from guideline to decision table.
- **Adversarial Agents:** A specialized "Red Team" AI attacks each step, hunting for "silent killers":
  - *What if they are out of the treatment?*
  - *Does this rule contradict Page 42?*
  - *Are terms unclear?*
- **Consensus Gate:** No code generation unless both pipelines exhibit behavioral equivalence

Pipeline of LLM prompts

Determine supplies and equipment

Determine local environmental shifters: Rural, “Malaria nearby,” stockout of malaria rapid test, etc.

Divide the manual into specific facts and “species” of fact: Determine order of modules (cough, fever, etc); determine when to do a test or ask a questions; clinically relevant cutpoints, decision rules; integrative issues (drug interactions, etc).

Look for “rough edges” in the manual and wait for MOH clarifications

Digesting a manual

Create a list of facts

Create a table of symbols (facts, tests, diagnoses, treatments, etc.)

- Building off of WHO's crisis care digital adaptation kit, whose dictionary has most of the relevant terms (along with FHIR-compliant codes for drugs, diagnoses, etc.)

Turn all continuous variables into Boolean variables used in decisions.

Create rules for when modules get added to the queue and the order of execution in the queue (including possibly skipping the queue for emergency referrals.

Create the clinical decision logic for each module

SCHEMAS

Markdown format to describe the pipeline from PDF manual to DMN and then XLSForm

DMN: What subset of logic

Patient data

Medical record = log file

Snippet of text and source and hash

Symbols table = Dictionary mapping concepts and definitions &

- Phrase bank which translates "malaria" into Hindi or Swahili
- Keywords like "malaria" or "temp_C" or "high_fever" ex.temp = exam to take temperature or dx.malaria
- Predicate tableturns ontinuous and categorical data into true/false: `if temp_C > 37.5 then has_fever = true , if muac=red then muac_red= true`

MOH approval of the clinical logic

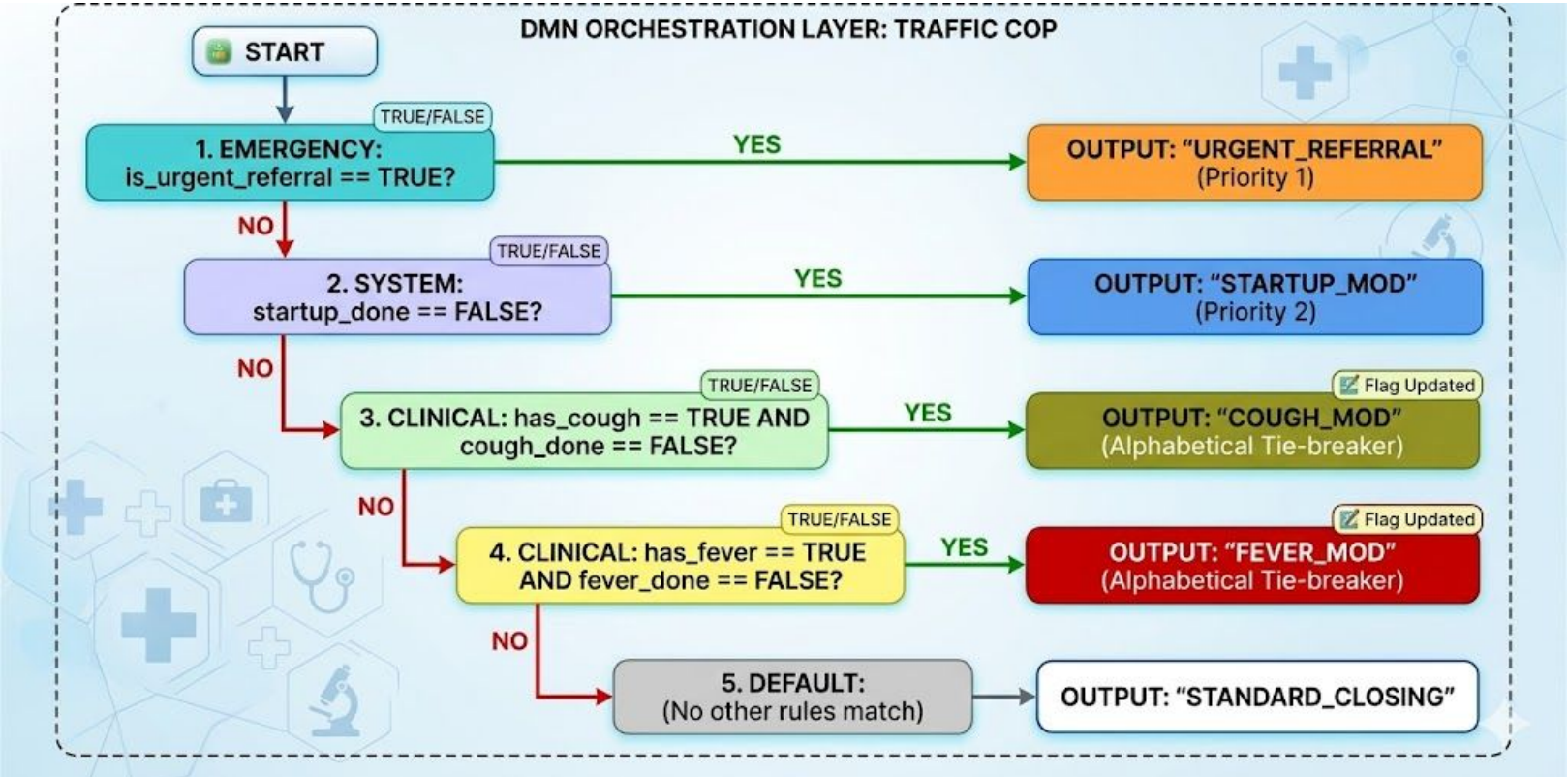
Turn the clinical logic into artifacts for clinicians to review

- A brief narrative
- a decision table
- A flowchart

Wait for MOH clinical approval

Flowchart of the traffic cop: Output = next module

Note: Runs after every module, priority order



Traffic cop fragment, priority order

	Start	**has_fever & not fever_mod_done**	**has_cough & not cough_mod_done**	**urgent_referral**	**OUTPUT (that is, next module to run)**
Urgent	-	-	-	True	Urgent referral (priority = 1)
At start	True	-	-	-	Opening_mod (priority = 2)
Fever	-	True	-	-	Fever_mod (priority = 4. It is lower priority than cough_mod solely because it is later alphabetically)
Cough	-	-	True	-	Cough_mod (priority = 3)
Closing	-	-	-	-	Closing_mod (priority = 5. Because the closing module is the lowest priority, it only runs when the queue is empty)

Notes:

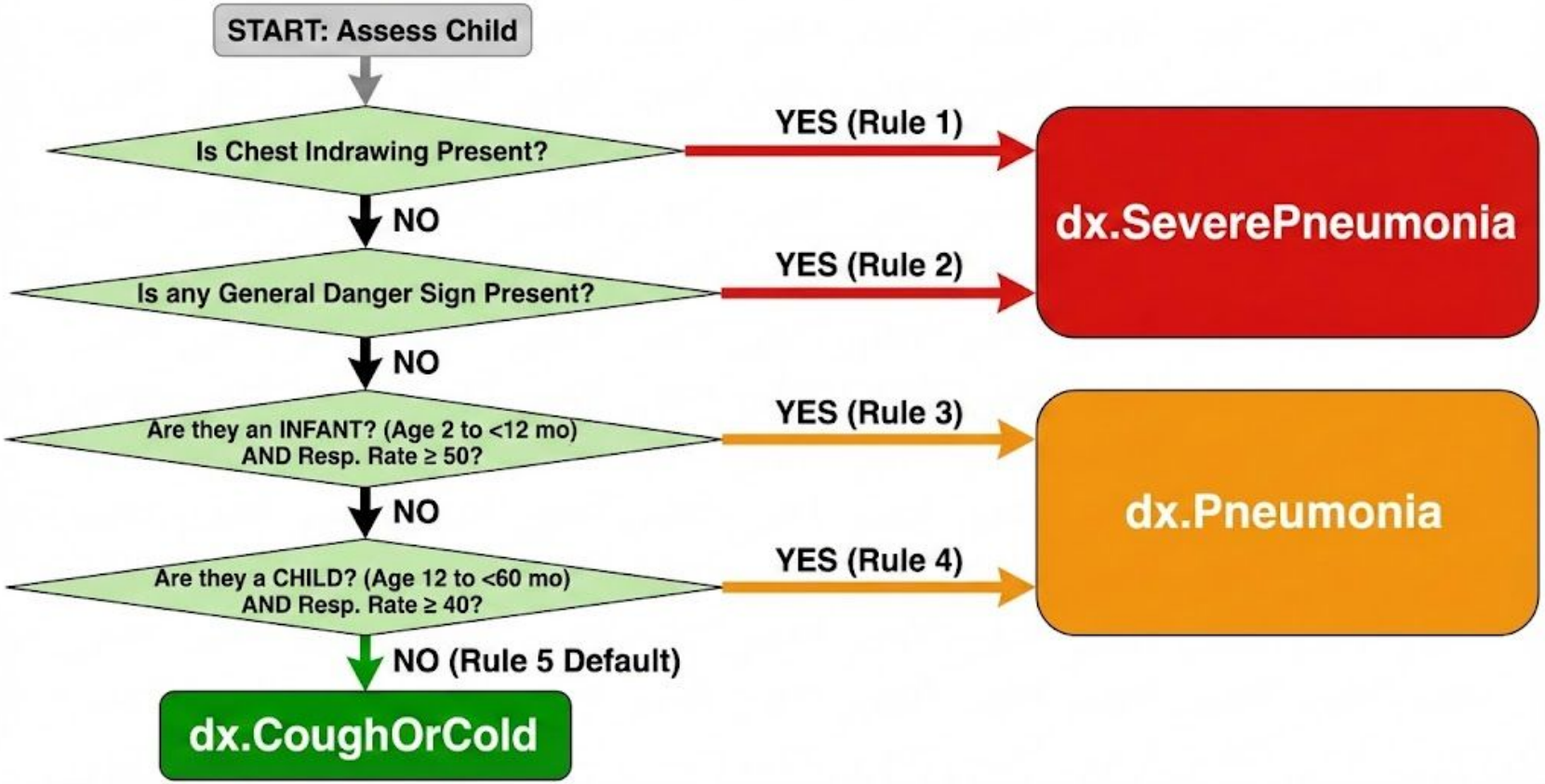
- Priority is metadata for each output. The AIs set priority, probably just before they write the DMN tables. Decision tables shown to MDs must show priority for each option.
- Any module can set has_fever, has_cough, etc. to True.
- The last thing module X does is set X_mod_done =True; e.g the fever module sets fever_mod_done = True.

The rhythm of a patient

Queue starts with

- Set at_start = True
- Traffic cop picks the next module in priority order
  - Emergency referral (if present) has highest priority
- Until traffic cop has no more modules to do & ends

Clinical Decision Flowchart for Child Respiratory Illness



A sample clinical decision table that a large language model distilled from a CHW training manual

Rule #	Description	Age (months)	Resp. Rate	Chest Indrawing?	Danger Sign?	OUTPUT: Classification
1	Chest indrawing is present	-	-	**TRUE**	-	SeverePneumonia
2	A general danger sign is present	-	-	FALSE	**TRUE**	SeverePneumonia
3	Fast breathing in an infant	>= 2 and < 12	**>= 50**	FALSE	FALSE	Pneumonia
4	Fast breathing in a child	>= 12 and < 60	**>= 40**	FALSE	FALSE	Pneumonia
5	Default case (no severe signs)	-	-	FALSE	FALSE	CoughOrCold

- This "DMN" table for children with a cough was generated solely by an LLM
- We can easily translate this table into the XLSForm that CHT uses
- MDs can review tables like this one
  - And into a flowchart for clinicians to review
- At end: set cough_mod_done = true and return to the traffic cop

For any module

Cough table asks about a combination of respiration rate and age

Implies: respiration rate & age are in the data before we get to this table

Turn continuous or categorical into true/false (Boolean or dummy) variables:

- if age_mo < 12 then age_lt_12_mo = true (MD sees "Age < 12 months")
- if rr_min >= 50 then rr_ge_50_min = true ("RR ≥ 50 / min.")
- if rr_min >= 40 then rr_ge_40_min= true

For each measure, we handle missing data, as in:

- if muac = missing then muac_missing = true

What inputs does each DMN require?

A DMN requires some boolean inputs.

Each boolean input requires (and implies)

1. A yes/no question or exam (e.g., Is there blood in stool?)

2. A yes/no data from the medical record (not yet implemented, and not a priority)

3. boolean, Categorical or continuous data from question, exam or medical record + some logic to create yes/no. (e.g., "Ask age in months or years") converts to age in months converts to age_le_12_mo or whatever.

So the one artifact created by the LLM is to draw out these implications and format them so the compiler can output the appropriate XLSForm.

For example: a cough diagnosis table might have inputs:
p_age_2_59m p_fast_breathing p_chest_indrawing p_danger_sign

So implies logic

* p_age_2_59m = true if age_mo >= 2 & age_mo <=59

* p_fast_breathing = F(measurement of respiration rates / minute, age)

* p_danger_sign = true if any of a bunch of earlier observations or questions on convulsions, etc. are true

In turn, that logic implies data collection (if not already collected):

- ask or find in medical record: age_mo
- measurement of respiration rate
- observation of chest indrawing
- observation of all the general danger signs

One intermediate artifact is the source of each variable:

- Some data are from the opening module: age_mo and observation of all the general danger signs. So no need to collect in later modules.
- Some data may be collected only in one module, so always collect for that DMN
- Some data may be already known if they went to a particular earlier module. So need to check "If temp_C = missing then ex_temp_C" where ex_temp_C is instructions to the CHW to take temperature.

There is a standard list of variable names

q.age_mo "What is you child's age (in months)?"

ex.temp_C "Use the thermometer to take temperature"

Many (but not all) are in the dictionary emett circulated.

You can add to that dictionary following the naming conventions.

Possible example of a fragment of a patient's data: {age_mo : 32, temp_C : 38.1, cough = True ,... }

- Note: product, testing and medical teams should create the real format

If the manual does not cover a stock-out, mention that fact

SAMPLE Treatment table for Cough module

Diagnosis	CHW Has amox	Treatment	Dosing table
Pneumonia	True	tx_amox	dose_amox_pneumonia
Pneumonia	False	ALERT: Manual does not cover this case	
SeverePneumonia	-	refer_clinic_today	N/A
coughCold	-	Go home and rest	N/A

Input	Type
age_months	number
diagnosis_pneumonia	boolean
danger_sign	boolean
chest_indrawing	boolean

Output	Type
amoxicillin_single_dose_mg	number
doses_per_day	number
treatment_days	number
tablets_250mg_per_dose	number

DMN Decision Table

Decision Name: `amoxicillin_dose_pneumonia_child`

Hit Policy: `UNIQUE`

Inputs

Outputs

DMN Table

Sample DMN dosing table: Metadata

Decision

Decision Name:
`amoxicillin_dose_pneumonia_child`
Hit Policy: `UNIQUE`

Outputs

Output	Type
tablets_250mg_per_dose	number
doses_per_day	number
treatment_days	number

Inputs

Input	Type	Description
age_months	number	Age in months

LLM generates and shares with testeing team: a table with conversions from continuous to boolean, such as: if age_mo <= 6 then age_le_6 _mo = true

Sample DMN dosing table
Fires if tx_amoxicillin= true

Rule	Age_le_6 _mo	tablets_250mg_per_dose	doses_per_day	treatment_days
1	true	1	2	5
2	false	2	2	5
3	missing	…		

Traffic cop 1: Add modules to queue (runs after every module)
Hit policy: Multiple hits allowed
Inputs: Start (boolean)+symptoms that trigger a module (such as Fever, etc.)+queue of modules
Output: Queue of modules

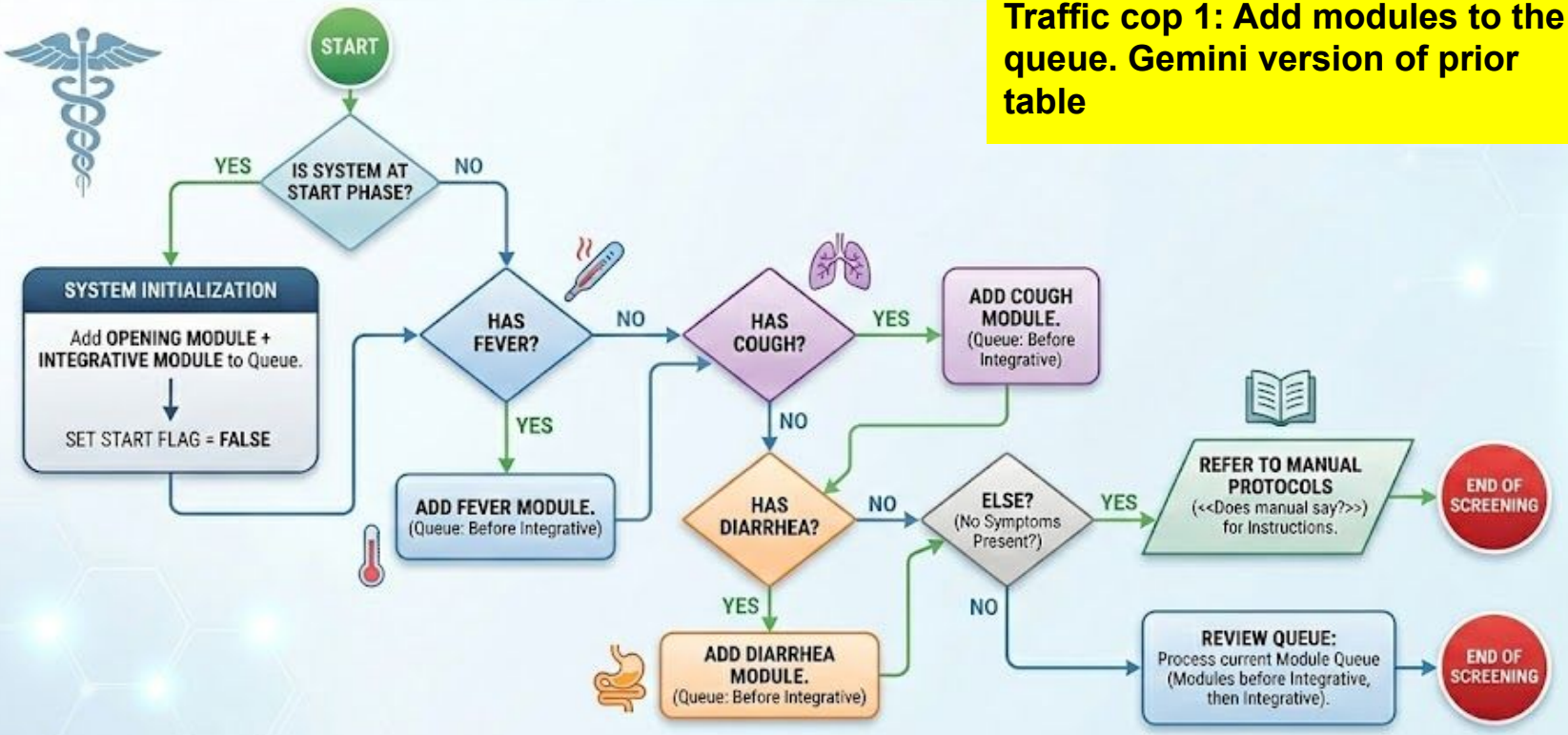
	Start?	**Has fever?**	**Has cough?**	**Has diarrhea?**	**Results**
At start	yes	any	any	any	Add opening module + integrative module Set start = false
Fever	no	yes	any	any	Add fever module at end of queue, but before integrative
Cough	no	any	yes	any	Add cough module at end of queue, but before integrative
Diarrhea	no	any	any	yes	Add diarrhea module at end of queue, but before integrative
Else	no	no	no	no	End this decision table (unless at start, in which case see manual)

Traffic cop 1: Add modules to queue (runs after every module)
Hit policy: Multiple hits allowed Inputs: Start (boolean)+symptoms that trigger a module (such as Fever, etc.)+queue of modules Output: Queue of modules

	Start?	Has fever?	Done fever module	Has cough?	Done diarrhea?	Has diarrhea?	Done diarrhea?	Results
At start	yes	any		any	any	any	any	Add opening module + integrative module Set start = false
Fever	no	yes	No	any	any	any	any	Add fever module at end of queue, but before integrative
Cough	no	any	any	yes	no	any	any	Add cough module at end of queue, but before integrative
Diarrhea	no	any	any	any	any	yes	no	Add diarrhea module at end of queue, but before integrative
Else	no	no	any	no	any	no	any	End this decision table (unless at start, in which case see manual)

SYMPTOM SCREENING AND MODULE QUEUING PROCESS (V1.2)

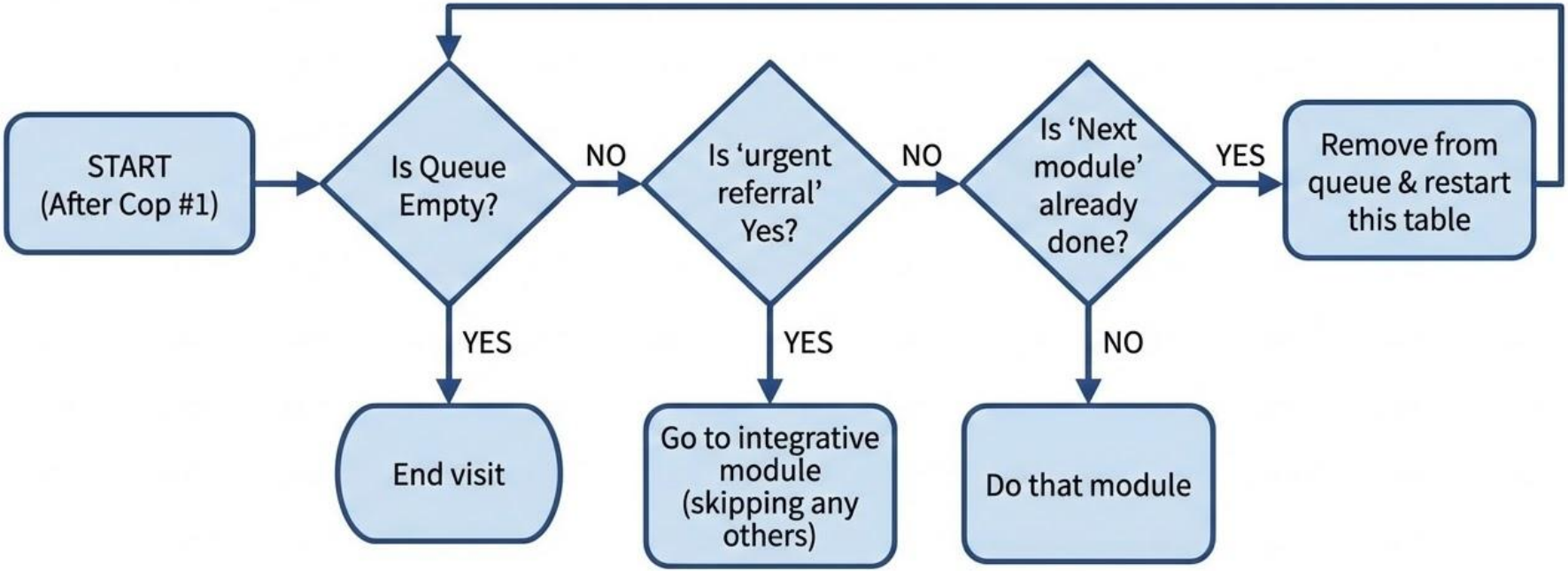
Traffic cop 1: Add modules to the queue. Gemini version of prior table



Traffic cop 2: pick next module to run (runs after every module, after cop #1)

Gemini drew this flowchart based on the decision table

Flowchart representation of the decision table logic



Translate all CHW-facing text

Two separate AIs translates all phrases into the requested languages, including the “How do I?” guidance for giving exams, providing treatments, etc. They

- Avoid jargon
- Use locally appropriate terms.

A third AI back-translates

A final AI reviews and improves the translations

The resulting phrase book is shared with clinicians and CHWs in a form that they can easily edit

The software uses only phrases that clinicians and CHWs have approved

Traffic cop:
Hit policy = mutually exclusive
Inputs = queue of modules + list of completed modules
Output = queue of modules + Done (boolean)+next module (e.g., starting module, fever, integrative)

	Queue empty	**Urgent referral**	**Next module already done?**	**Result**
Visit done	Yes	Any	Any	End visit
Urgent referral	No	Yes	Any	Go to integrative module (skipping any others)
Next module in queue	No	No	Yes	Remove from queue & restart this table
Next module in queue	No	No	No	Do that module

Mermaid of cough DMN

```
%%{init: {'theme': 'base', 'themeVariables': { 'fontSize': '24px', 'fontFamily': 'Helvetica', 'primaryColor': '#fff', 'edgeLabelBackground':'#ffffff', 'tertiaryColor': '#f4f4f4'}}}%%
graph TD
    %% --- STYLING DEFINITIONS ---
    classDef questionNode fill:#d4edda,stroke:#28a745,stroke-width:3px,color:black,font-size:20px,font-weight:bold;
    classDef severeOutcome fill:#dc3545,stroke:#721c24,stroke-width:4px,color:white,font-size:26px,font-weight:bold,rx:15,ry:15;
    classDef pneumoniaOutcome fill:#fd7e14,stroke:#ba5e0f,stroke-width:4px,color:white,font-size:26px,font-weight:bold,rx:15,ry:15;
    classDef mildOutcome fill:#28a745,stroke:#155724,stroke-width:4px,color:white,font-size:26px,font-weight:bold,rx:15,ry:15;
    classDef startNode fill:#e2e6ea,stroke:#333,stroke-width:2px,font-size:20px;
    linkStyle default font-size:20px, stroke-width:3px;

    %% --- FLOWCHART LOGIC ---
    Start(("START: Assess Child")):::startNode --> Q1

    Q1{"Is Chest Indrawing<br>Present?"}:::questionNode
    Q1 -- YES (Rule 1) --> Severe[dx.SeverePneumonia]:::severeOutcome
    Q1 -- NO --> Q2

    Q2{"Is any General<br>Danger Sign Present?"}:::questionNode
    Q2 -- YES (Rule 2) --> Severe
    Q2 -- NO --> Q3

    Q3{"Are they an INFANT?<br>(Age 2 to <12 mo)<br>AND<br>Resp. Rate ≥ 50?"}:::questionNode
    Q3 -- YES (Rule 3) --> Pneumonia[dx.Pneumonia]:::pneumoniaOutcome
    Q3 -- NO --> Q4

    Q4{"Are they a CHILD?<br>(Age 12 to <60 mo)<br>AND<br>Resp. Rate ≥ 40?"}:::questionNode
    Q4 -- YES (Rule 4) --> Pneumonia
    Q4 -- NO (Rule 5 Default) --> Mild[dx.CoughOrCold]:::mildOutcome
```

Compile the clinical logic into executable code

For example, translate the clinical logic into XLSForm for CHT

But can we trust AI output?

The Verification Stack for Zero-Trust Engineering

- **Adversarial Extraction:** "Red Teams" hunting for logic gaps at each step of creating the clinical logic and code
- **Human Sovereignty:** Mandatory sign-offs of clinical logic & of translations
- **Traceability:** Cryptographic links to source text
- **Formal Verification:** Mathematical proofs of consistency, various quality standards
- **Differential Testing:** Synthetic patient simulations show the final code replicates the doctor-approved clinical logic

Adversarial extraction: "Red Teams" hunt for logic gaps at each step

Our pipeline uses LLMs to divide the manual into pieces and assembles the pieces into the clinical logic:

- Read manual PDF into structured text file
- Identify lists of supplies, decisions, possible diagnoses, possible treatments, etc.
- Create decision tables with the clinical logic
- Etc.

At each step, a separate AI has a "red team" prompt checking if each intermediate artifact is complete and hits is quality standards

A third AI adjudicates between the first output and the red team review (and cycles back to the red team, up to 3 times)

Sample quality standards for the red team

Consistency of DMN & its building blocks (unless clinicians approve an exception)

- All diagnostic equipment and supplies and all treatments mentioned in the Supply list are used in a Decision table, and conversely
- All variable names (symptoms, exams, drugs, etc.) are in the dictionary and the phrase book
- Rules exist to convert all categorical and continuous variables into the true/false variables used by DMN
- ...

Sample quality standards for the red team

- DMN decision tables
  - Every DMN decision row is reachable
  - Only true/false inputs
  - no FEEL arithmetic
  - no hidden calculations inside DMN
  - explicit ELSE row
  - explicit hit policy
  - outputs that are enumerated and reviewable
  - integrative logic separate from module-local logic when possible
- XLSForm
  - Every rule in XLSForm matches the doctor-approved rule on boundary cases
  - Every output code in XLSForm has a corresponding DMN output value
- etc.

Human Sovereignty & Impact

AI generates the draft clinical logic, but **clinicians own the decision.**

- **Mandatory Human Gates**
  - **Ambiguity Resolution:** MOH must clarify manuals that the AI does not consider complete
    i. e.g., “The manual does not define a rule if a stockout of…”
  - **Logic Review:** MOH clinicians approve the clinical logic presented in **Plain English** and **Flowcharts** (not code)
    i. For updates, the system highlights where the update modifies the logic
  - **Translation Sign-off:** Local staff approve exact message wording
- **The Result:** Code that is clinically credible and MOH-approved

Mathematical Proof of Consistency with Z3 prover

Use formal prover to show the decision rules have desirable properties

- All paths end in a clinical diagnosis
- No patients have mutually exclusive diagnoses or treatments
- No cycles
- Plans for all forms of stockouts
- etc.

Sample quality standards: Universal rules

Unless the manual or a clinician overrides:

- Adding a diagnosis, adding a general danger sign, or worsening a symptom (e.g., a higher fever)
    - Never reduces referral intensity
        - e.g., never replaces hospital referral with clinic or clinic with home care
    - Never lengthens time to follow-up

We also ensure usability: Maximum # of clicks and of screens

Traceability and Replicability

Traceability: Every rule and line of code is linked to a specific page and text snippet from the source manual(s)

Replicability: We cryptographically sign and freeze the clinical logic after MOH approval. All code and libraries that validate the clinical logic and then compile the logic into code such as XLSForm live in a single container with a unique hash.

Everything after clinical sign-off is deterministic (no AI), so is bit-by-bit replicable

z3 also generates thousands of Synthetic Patient Profiles.

- All one-diagnosis cases & clinical endpoints
- **Comorbidities:** Both common and rare combinations of symptoms
  - All two-diagnosis cases
  - critical three-way interactions

- Generate **Stress Tests:**
  - **Boundary Triples:** Temp = 37.4°C, 37.5°C, 37.6°C.
  - **Missingness:** Handles broken instruments, stock-outs, skipped questions.

Compare output from doctor-approved DMN and XLSForm that runs on CHWs' phones

For each synthetic patient, compare:

- Questions & exams
- Diagnoses
- Treatment including
  - Drugs
  - Dosing
  - Follow-up visit timing
  - Referral urgency
  - Stockout substitutions
  - Conflict resolution across diagnoses or treatments

After we compile the clinical logic into executable code

For example, for CHT the system translates the clinical logic into XLSForm

We verify that thousands of simulated patients have identical diagnosis and treatment when run through:

A. The decision logic the MOH approved
B. The flowchart made from that logic
C. The executable code (such as XLSForm) compiled from that logic

Input to Test harness

1. Decision tables
2. Predicate Table for converting continuous data from patient data (age, temperature, etc.) to boolean variables used in the DMN decision tables

infant_fast_breathing = (age_mo > 2) AND (age_mo <=12 mo) AND (resp_rate ≥ 50)

child_fast_breathing = (age_mo >12 mo) AND (resp_rate ≥ 40)

Differential Testing

- **Mutation** testing: When we purposely modify any rules or patients, we require the system to identify 100% of such discrepancies
-

Quality checks: Reliability

Test-retest reliability: Give same manual and get clinically identical output

Inter-rater reliability: Use different LLMs for different tasks and get clinically identical output

Note:

- Reliability does not prove validity,
- but *failure* of reliability proves at least one run is wrong

Quality checks

Humans use the system to ensure

- Sensible logic flow
- Clear language
- Reasonable # of clicks and of screens
- etc.

Validation of this method (planned)

Validation of our pipeline against independent oracles

1. **Legacy Replication:** Our pipeline leads to identical clinical decisions as existing hard-coded CHW software derived from a manual
2. **WHO Gold Standard:** For three WHO CHW manuals, we hand-coded decision tables and had MDs verify each rule. Our system's decision logic reproduced the MD-approved rules.
3. **Stress tests with synthetic adversarial manuals:** We created complex synthetic manuals with known ground truth.
   a. The system reproduces the known correct behavior.
   b. When we input incomplete or internally inconsistent manuals, the system correctly identifies the issues

Field evidence #1: CHWs & Artificial patients

Recruit CHWs for a weekend

Give synthetic (artificial) patients

Compare adherence to guidelines with and without CHW Navigator

- Note: Adherence will be artificially high in both conditions

Field evidence #2: CHWs & real patients with MD oversight

CHWs treat real patients with a nurse or doctor watching to ensure no harms

Compare adherence to guidelines with and without CHW Navigator

- Note: Adherence will be artificially high in both conditions

Innovation Summary

- **Summary:**
  - **From:** Manual Transcription → **To:** Verified Compilation.
  - **From:** "Black Box" Logic → **To:** Traceable Provenance.
  - **From:** Post-Deployment Debugging → **To:** Pre-Deployment Clinical review+ Mathematical Proof.
- **Final Impact:** A generalizable framework for producing auditable, reproducible, and trustworthy decision support in low-resource settings.